

CENTRO UNIVERSITÁRIO LEONARDO DA VINCI — UNIASSELVI

Engenharia de Software — 7º período



PADRÃO DE CODIFICAÇÃO DA EQUIPE

Aplicativo de Detecção de Tentativas de Phishing por Correio Eletrônico — Etapa 9 do Roteiro de Execução (Fase 2)

Projeto: Aplicativo de Detecção de Tentativas de Phishing por Correio Eletrônico

Versão: 1.0

Data de emissão: 25/09/2026

Equipe: Ethan Buttazzi (projetista, pesquisador e implementador). Sem revisão por pares — o checklist da seção 6 substitui a revisão cruzada.

1. Linguagem e nomenclatura

Convenções oficiais da Microsoft para C#/.NET:

- PascalCase — classes, interfaces (prefixo `I`), métodos, propriedades, namespaces.
- camelCase — variáveis locais e parâmetros.
- _camelCase — campos privados (prefixo `\_`).
- Nomes em português para conceitos de domínio (`MensagemRecebida`, `TemplateEmpresa`), consistentes com o Documento de Requisitos; termos técnicos de infraestrutura podem seguir a convenção da própria biblioteca (ex.: `Repository`, `Service`) quando fizer sentido.

2. Organização do projeto

Pastas por camada, refletindo desde já a arquitetura que será formalizada na Etapa 15:

/Domain

/Application

/Infrastructure

/Presentation

Um namespace por camada. Nenhuma classe de `Domain` referencia `Infrastructure` ou `Presentation`.

3. Princípios SOLID aplicados

Princípio	Como se aplica aqui
S — Responsabilidade única	Cada classe/serviço faz uma coisa (ex.: `ValidadorDeHyperlink` só valida; não notifica, não grava).
O — Aberto/fechado	Novas checagens de RF-07 devem poder ser adicionadas sem alterar as já existentes (cada checagem isolada em seu próprio método).
L — Substituição de Liskov	Qualquer implementação de uma interface de repositório substitui outra sem quebrar quem a usa (ex.: implementação real e uma implementação em memória para teste).
I — Segregação de interface	Interfaces pequenas e específicas — um repositório não carrega métodos que só outro usa.
D — Inversão de dependência	`Domain` declara as interfaces de repositório de que precisa; `Infrastructure` as implementa e depende do

Princípio	Como se aplica aqui
	domínio, nunca o contrário.

4. Controle de versão

- Repositório Git único.
- Um branch por funcionalidade: `feature/f-XX-nome-curto`, referenciando F-01 a F-08 (Documento de Requisitos, seção 7).
- Commits pequenos e descritivos — um por incremento funcional verificável.
- Mesclagem à `main` somente quando o card atinge a Definition of Done (Documento de Requisitos, seção 12).

5. Segurança

- Nenhuma credencial, senha, client secret ou token OAuth2 fica hardcoded no código (regra herdada diretamente da DoD).
- Segredos ficam fora do controle de versão (variáveis de ambiente ou arquivo de configuração local ignorado pelo Git).

6. Checklist de auto-revisão (antes de mesclar à main)

- ☐ Nomenclatura segue a seção 1.
- ☐ A classe está na pasta/camada certa (seção 2) e não importa nada de uma camada "de fora para dentro" do domínio.
- ☐ Nenhum princípio SOLID da seção 3 foi violado deliberadamente sem justificativa.
- ☐ Nenhum segredo hardcoded.
- ☐ Commit pequeno, com mensagem descritiva.

Controle de Versão do Documento

Versão	Data	Item alterado	Razão e impacto
1.0	25/09/2026	Emissão inicial	Elaborado para a Etapa 9 do Roteiro de Execução (Fase 2 — Decisões técnicas de base), seguindo convenções .NET e princípios SOLID.